HCP KubeVirt L2 Network Isolation PoC — Build Guide

This guide deploys two Hosted Control Plane (HCP) clusters with KubeVirt worker VMs on a single Hetzner dedicated server running OpenShift SNO 4.22. Each hosted cluster gets its own isolated L2 network via OVS bridges, proving that OCP-Virt + HCP can replace OpenStack VLANs for inter-cluster network isolation.

> **SNO-specific sections** are marked with **SNO**. On a multi-node management cluster, some workarounds (hairpin NAT, maxPods, NodePool update strategy) are unnecessary.

Prerequisites

Server

Model	CPU	RAM	Storage	Price
AX42/AX52	AMD Ryzen 7 7700 (8c/16t)	64 GB DDR5	2x 1TB NVMe	~55 EUR/month

SNO: Minimum 8c/16t. A 4c/8t server does not have enough CPU request headroom for SNO + 5 operators + 2 HCP control planes (~10.4 vCPU of requests).

DNS Records (Cloudflare)

Create these DNS records **before starting**. All point to your server's public IP (e.g., `157.180.56.29`).

Management cluster (public DNS):

Type	Name	Content	Proxy
A	`api.ocp4.lapthorn.me`	157.180.56.29	DNS only
A	`*.apps.ocp4.lapthorn.me`	157.180.56.29	DNS only

Hosted cluster bootstrap/control-plane routes (public DNS):

These are needed so the management cluster's IngressController shards can serve routes for HCP services (OAuth, Konnectivity, Ignition). They resolve to the server's public IP because the

shard IngressControllers get MetalLB VIPs on the data networks, but the routes are initially accessed via the public-facing default IngressController during bootstrap.

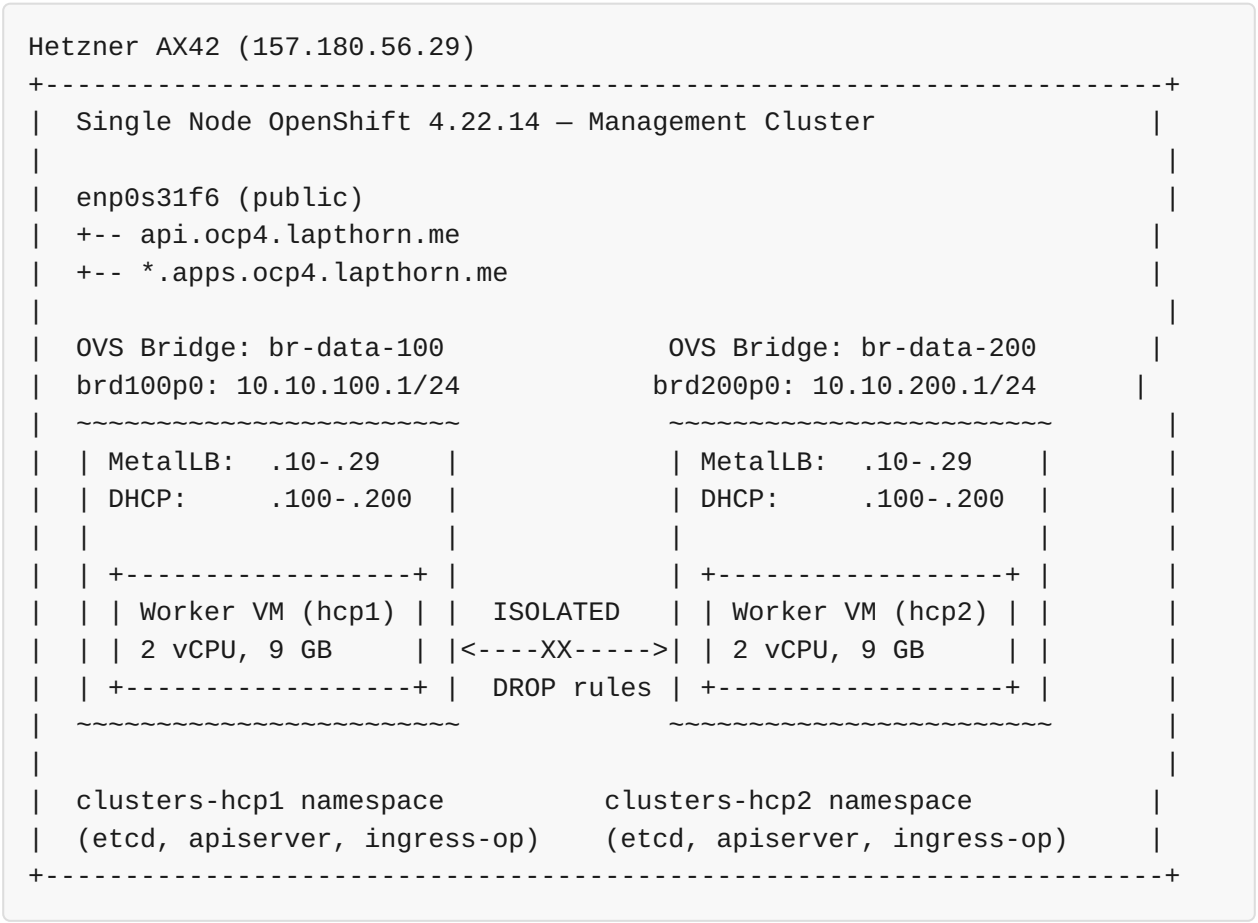
Type	Name	Content	Proxy
A	`*.hosted.hcp1.lapthorn.me`	157.180.56.29	DNS only
A	`*.hosted.hcp2.lapthorn.me`	157.180.56.29	DNS only

> **Note:** The `api.hcpX` and `apps.hcpX` domains are resolved by dnsmasq on the SNO node's data network interfaces (not public DNS), so they do NOT need Cloudflare records.

Tools (on your workstation)

- `oc` CLI (matching your OCP version)
- `hcp` CLI (from MCE — extract from the MCE operator image or `oc get ConsoleCLIDownload`)
- `openshift-install` (matching your OCP version)
- SSH key for the server
- Pull secret from console.redhat.com

Architecture



IP Address	Purpose
10.10.X00.1	Gateway (OVS bridge interface on SNO)
10.10.X00.10	MetalLB: bootstrap IngressController shard
10.10.X00.11	MetalLB: kube-apiserver LoadBalancer
10.10.X00.12	Apps ingress (secondary IP + iptables DNAT to router NodePort)
10.10.X00.100-.200	DHCP range for worker VMs

Part 1: Install SNO

Install OpenShift SNO 4.22 using `aicli` (assisted installer CLI) or the agent-based installer. This guide assumes SNO is already installed and you have `kubeconfig.ocp4`.

Verify:

```
export KUBECONFIG=~/sno-install/kubeconfig.ocp4
oc get nodes
oc get clusterversion
```

Part 2: Install Operators

Install these five operators. Order matters — install MetalLB and NMState first as other components depend on them.

2.1 Operator Subscriptions

SNO: Channel versions are specific to OCP 4.22. Check for your version.

```
# metallb-operator — namespace: metallb-system
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: metallb-operator
  namespace: metallb-system
spec:
  channel: stable
  name: metallb-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace
---
# kubernetes-nmstate-operator — namespace: openshift-nmstate
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: kubernetes-nmstate-operator
  namespace: openshift-nmstate
spec:
  channel: stable
  name: kubernetes-nmstate-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace
---
# hco-operatorhub (OpenShift Virtualization) — namespace: openshift-cnv
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: hco-operatorhub
  namespace: openshift-cnv
spec:
  channel: stable
  name: kubevirt-hyperconverged
  source: redhat-operators
  sourceNamespace: openshift-marketplace
---
# lvms-operator — namespace: openshift-storage
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: lvms-operator
  namespace: openshift-storage
spec:
  channel: stable-4.22          # NOT "stable" — must match OCP minor version
  name: lvms-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace
---
# multicluster-engine (MCE) — namespace: multicluster-engine
apiVersion: operators.coreos.com/v1alpha1
```

```
kind: Subscription
metadata:
  name: multicluster-engine
  namespace: multicluster-engine
spec:
  channel: stable-2.17          # NOT "stable-2.8"
  name: multicluster-engine
  source: redhat-operators
  sourceNamespace: openshift-marketplace
```

> **Important:** Each operator needs a `Namespace` and `OperatorGroup` created first. MetalLB's OperatorGroup must use `AllNamespaces` scope (empty `spec: {}`). `OwnNamespace` mode fails with "OwnNamespace InstallModeType not supported".

2.2 Create Operator Instances

After all operators report `Succeeded`, create their CRs:

```
# MetalLB instance
oc apply -f - <<EOF
apiVersion: metallb.io/v1beta1
kind: MetalLB
metadata:
  name: metallb
  namespace: metallb-system
spec: {}
EOF

# NMState instance
oc apply -f - <<EOF
apiVersion: nmstate.io/v1
kind: NMState
metadata:
  name: nmstate
spec: {}
EOF

# HyperConverged (CNV)
oc apply -f - <<EOF
apiVersion: hco.kubevirt.io/v1beta1
kind: HyperConverged
metadata:
  name: kubevirt-hyperconverged
  namespace: openshift-cnv
spec: {}
EOF

# MultiClusterEngine
oc apply -f - <<EOF
apiVersion: multicluster.openshift.io/v1
kind: MultiClusterEngine
metadata:
  name: multiclusterengine
spec:
  targetNamespace: multicluster-engine
EOF
```

2.3 LVM Storage

If your second NVMe has residual filesystems (e.g., iso9660 from the discovery ISO), wipe it first:

```
# SSH to the node
ssh core@157.180.56.29
sudo wipefs -a /dev/nvme1n1
```

Then create the LVMCluster:

```
apiVersion: lvm.topolvm.io/v1alpha1
kind: LVMCluster
metadata:
  name: lvms-vg1
  namespace: openshift-storage
spec:
  storage:
    deviceClasses:
    - name: vg1
      default: true
      deviceSelector:
        paths:
        - /dev/nvme1n1
      thinPoolConfig:
        name: thin-pool-1
        sizePercent: 90
        overprovisionRatio: 10
```

2.4 KubeletConfig — Increase maxPods

SNO: HCP control planes create many pods. The default limit of 250 is too low. This triggers a node reboot.

```
apiVersion: machineconfiguration.openshift.io/v1
kind: KubeletConfig
metadata:
  name: max-pods-500
spec:
  kubeletConfig:
    maxPods: 500
  machineConfigPoolSelector:
    matchLabels:
      pools.operator.machineconfiguration.openshift.io/master: ""
```

Wait for the MachineConfigPool to finish updating:

```
oc get mcp master -w
```

Part 3: Enable IP Forwarding

This allows traffic from the data networks to be forwarded (required for NAT and inter-network routing). Does NOT require a reboot.

```
oc patch network.operator cluster --type=merge -p '{
  "spec": {
    "defaultNetwork": {
      "ovnKubernetesConfig": {
        "gatewayConfig": {
          "ipForwarding": "Global",
          "routingViaHost": true
        }
      }
    }
  }
}'
```

Part 4: Create Data Network Infrastructure

4.1 OVS Bridges (NMState NNCPs)

Create two OVS bridges — one per hosted cluster. These are the L2 segments that replace VLANs.

> **Critical:** IPs go on `type: ovs-interface` entries, NOT on `type: ovs-bridge`. Do NOT list bridge ports — OVN adds patch ports dynamically which cause NMState verification failures.

```
apiVersion: nmstate.io/v1
kind: NodeNetworkConfigurationPolicy
metadata:
  name: br-data-100
spec:
  nodeSelector:
    node-role.kubernetes.io/master: ""
  desiredState:
    interfaces:
    - name: br-data-100
      type: ovs-bridge
      state: up
      ipv4:
        enabled: false
    - name: brd100p0
      type: ovs-interface
      state: up
      ipv4:
        enabled: true
        dhcp: false
        address:
        - ip: 10.10.100.1
          prefix-length: 24
---
apiVersion: nmstate.io/v1
kind: NodeNetworkConfigurationPolicy
metadata:
  name: br-data-200
spec:
  nodeSelector:
    node-role.kubernetes.io/master: ""
  desiredState:
    interfaces:
    - name: br-data-200
      type: ovs-bridge
      state: up
      ipv4:
        enabled: false
    - name: brd200p0
      type: ovs-interface
      state: up
      ipv4:
        enabled: true
        dhcp: false
        address:
        - ip: 10.10.200.1
          prefix-length: 24
```

> **Why short names?** Linux limits interface names to 15 characters. `br-data-100-port0` (17 chars) fails silently. Use `brd100p0` (8 chars).

4.2 OVN Localnet Mappings

Map the OVS bridges to OVN localnet names:

```
apiVersion: nmstate.io/v1
kind: NodeNetworkConfigurationPolicy
metadata:
  name: ovn-localnet-mappings
spec:
  nodeSelector:
    node-role.kubernetes.io/master: ""
  desiredState:
    ovn:
      bridge-mappings:
      - bridge: br-data-100
        localnet: localnet-100
        state: present
      - bridge: br-data-200
        localnet: localnet-200
        state: present
```

4.3 ClusterUserDefinedNetworks (CUDNs)

Attach the localnet networks to the HCP namespaces. Each CUDN is scoped to one namespace.

> **Critical:** `ipam.mode: Disabled` is mandatory. If you set subnets, OVN assigns IPs and injects cluster DNS (172.30.0.10) into the VMs. The VMs can't reach 172.30.0.10 from the isolated data network, so DNS breaks. With IPAM disabled, VMs use DHCP from dnsmasq which correctly sets DNS to the gateway.

```
apiVersion: k8s.ovn.org/v1
kind: ClusterUserDefinedNetwork
metadata:
  name: localnet-100
spec:
  namespaceSelector:
    matchLabels:
      kubernetes.io/metadata.name: clusters-hcp1
  network:
    topology: Localnet
    localnet:
      role: Secondary
      physicalNetworkName: localnet-100
      ipam:
        mode: Disabled
---
apiVersion: k8s.ovn.org/v1
kind: ClusterUserDefinedNetwork
metadata:
  name: localnet-200
spec:
  namespaceSelector:
    matchLabels:
      kubernetes.io/metadata.name: clusters-hcp2
  network:
    topology: Localnet
    localnet:
      role: Secondary
      physicalNetworkName: localnet-200
      ipam:
        mode: Disabled
```

> **Note:** The CUDN `network` spec is immutable. If you need to change it, scale NodePools to 0, delete the CUDN, recreate it, then scale back.

4.4 MetalLB IP Address Pools and L2 Advertisements

```
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: pool-100
  namespace: metallb-system
spec:
  addresses:
  - 10.10.100.10-10.10.100.29
---
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: pool-200
  namespace: metallb-system
spec:
  addresses:
  - 10.10.200.10-10.10.200.29
---
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: l2advert-100
  namespace: metallb-system
spec:
  ipAddressPools:
  - pool-100
  interfaces:
  - brd100p0
---
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: l2advert-200
  namespace: metallb-system
spec:
  ipAddressPools:
  - pool-200
  interfaces:
  - brd200p0
```

Part 5: Configure dnsmasq for DHCP and DNS

The SNO node already runs a system dnsmasq for cluster DNS. Add a drop-in config file for the HCP data networks. Do NOT run a second dnsmasq instance — port 53 will conflict.

SSH to the node:

```
ssh core@157.180.56.29
```

Create the drop-in config:

```
sudo tee /etc/dnsmasq.d/hcp-data.conf << 'EOF'
# DHCP for hcp1 network (10.10.100.0/24)
dhcp-range=set:net100,10.10.100.100,10.10.100.200,255.255.255.0,12h
dhcp-option=tag:net100,3,10.10.100.1
dhcp-option=tag:net100,6,10.10.100.1

# DHCP for hcp2 network (10.10.200.0/24)
dhcp-range=set:net200,10.10.200.100,10.10.200.200,255.255.255.0,12h
dhcp-option=tag:net200,3,10.10.200.1
dhcp-option=tag:net200,6,10.10.200.1

# DNS: hosted cluster bootstrap endpoints
address=/hosted.hcp1.lapthorn.me/10.10.100.10
address=/hosted.hcp2.lapthorn.me/10.10.200.10

# DNS: hosted cluster API endpoints
address=/api.hcp1.lapthorn.me/10.10.100.11
address=/api.hcp2.lapthorn.me/10.10.200.11

# DNS: hosted cluster apps endpoints
address=/apps.hcp1.lapthorn.me/10.10.100.12
address=/apps.hcp2.lapthorn.me/10.10.200.12

# Forward all other DNS queries to upstream
server=185.12.64.1
server=185.12.64.2

log-dhcp
EOF
```

> **Critical:** Do NOT add `interface=`, `bind-interfaces`, or `bind-dynamic` to this file. Any of these restricts dnsmasq's listening scope and breaks DNS on the primary interface (needed by cluster DNS and SSH SOCKS proxy). The DHCP tags (`set:net100`, `set:net200`) ensure DHCP is only served on the correct subnets.

Restart dnsmasq:

```
sudo systemctl restart dnsmasq
```

Part 6: NAT, Forwarding, and L2 Isolation (iptables)

6.1 Base Rules (systemd service)

Create a systemd service so these rules survive reboots:

```
sudo tee /etc/systemd/system/nat-data-networks.service << 'EOF'
[Unit]
Description=NAT and L2 isolation for HCP data networks
After=network-online.target
Wants=network-online.target

[Service]
Type=oneshot
RemainAfterExit=true

# ISOLATION: Block traffic between the two data networks
ExecStart=/usr/sbin/iptables -I FORWARD -s 10.10.100.0/24 -d 10.10.200.0/24 -j DROP
ExecStart=/usr/sbin/iptables -I FORWARD -s 10.10.200.0/24 -d 10.10.100.0/24 -j DROP

# FORWARDING: Allow each network to reach the internet
ExecStart=/usr/sbin/iptables -A FORWARD -s 10.10.100.0/24 -j ACCEPT
ExecStart=/usr/sbin/iptables -A FORWARD -s 10.10.200.0/24 -j ACCEPT
ExecStart=/usr/sbin/iptables -A FORWARD -m state --state RELATED,ESTABLISHED -j ACCEPT

# NAT: Masquerade outbound traffic
ExecStart=/usr/sbin/iptables -t nat -A POSTROUTING -s 10.10.100.0/24 ! -d 10.10.100.0/2
ExecStart=/usr/sbin/iptables -t nat -A POSTROUTING -s 10.10.200.0/24 ! -d 10.10.200.0/2

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable --now nat-data-networks.service
```

6.2 Apps Ingress DNAT and Hairpin NAT

SNO: These rules are needed because the hosted clusters' routers use NodePort (not LoadBalancer). The DNAT forwards apps traffic to the worker VM's router NodePort. The MASQUERADE rules fix the hairpin NAT problem where traffic from a VM goes to the MetalLB VIP on the same node and gets DNAT'd back to the same VM.

On a multi-node cluster, MetalLB would assign the VIP to a different node, avoiding the hairpin entirely.

First, add the apps VIPs as secondary IPs so the host accepts traffic for them:

```
sudo ip addr add 10.10.100.12/32 dev brd100p0
sudo ip addr add 10.10.200.12/32 dev brd200p0
```

Then add DNAT and hairpin MASQUERADE rules. **Replace `<VM_IP>` and `<NODEPORT>` with actual values** from your deployment (see Getting VM IPs and NodePorts):

```
# --- hcp1 ---
# DNAT: apps.hcp1 → worker VM's router NodePort
sudo iptables -t nat -A PREROUTING -d 10.10.100.12 -p tcp --dport 443 -j DNAT --to-dest
sudo iptables -t nat -A PREROUTING -d 10.10.100.12 -p tcp --dport 80  -j DNAT --to-dest
sudo iptables -t nat -A OUTPUT     -d 10.10.100.12 -p tcp --dport 443 -j DNAT --to-dest

# Hairpin MASQUERADE (SNO only)
sudo iptables -t nat -A POSTROUTING -s 10.10.100.0/24 -d <HCP1_VM_IP> -p tcp --dport <H
sudo iptables -t nat -A POSTROUTING -s 10.10.100.0/24 -d <HCP1_VM_IP> -p tcp --dport <H

# --- hcp2 ---
sudo iptables -t nat -A PREROUTING -d 10.10.200.12 -p tcp --dport 443 -j DNAT --to-dest
sudo iptables -t nat -A PREROUTING -d 10.10.200.12 -p tcp --dport 80  -j DNAT --to-dest
sudo iptables -t nat -A OUTPUT     -d 10.10.200.12 -p tcp --dport 443 -j DNAT --to-dest

sudo iptables -t nat -A POSTROUTING -s 10.10.200.0/24 -d <HCP2_VM_IP> -p tcp --dport <H
sudo iptables -t nat -A POSTROUTING -s 10.10.200.0/24 -d <HCP2_VM_IP> -p tcp --dport <H
```

Getting VM IPs and NodePorts

VM IPs change on every VM recreation. Get current values with:

```
# VM IPs (from management cluster)
oc get vmi -n clusters-hcp1 -o jsonpath='{.items[0].status.interfaces[0].ipAddress}'
oc get vmi -n clusters-hcp2 -o jsonpath='{.items[0].status.interfaces[0].ipAddress}'

# Router NodePorts (from inside the hosted clusters via SSH to the SNO node)
ssh core@<SERVER_IP>
KUBECONFIG=/tmp/hcp1-kubeconfig oc get svc -n openshift-ingress router-nodeport-default
  -o jsonpath='http:{.spec.ports[?(@.name=="http")].nodePort} https:{.spec.ports[?(@.na
```

> **Note:** These DNAT/MASQUERADE rules are NOT persisted in the systemd unit. You must re-add them after reboot or VM recreation. Consider extending `nat-data-networks.service` with a script that dynamically looks up VM IPs.

Part 7: IngressController Shards

Create IngressController shards so each hosted cluster's routes (OAuth, Konnectivity, Ignition) are served on the correct data network via MetalLB.

> **Critical:** Use `namespaceSelector`, NOT `routeSelector`. The HCP operator strips custom labels from routes during reconciliation, so `routeSelector` silently stops matching.

7.1 Create IngressControllers for hcp1 and hcp2

```
apiVersion: operator.openshift.io/v1
kind: IngressController
metadata:
  name: hcp1
  namespace: openshift-ingress-operator
spec:
  domain: hosted.hcp1.lapthorn.me
  replicas: 1
  namespaceSelector:
    matchLabels:
      kubernetes.io/metadata.name: clusters-hcp1
  endpointPublishingStrategy:
    type: LoadBalancerService
    loadBalancer:
      scope: Internal
      providerParameters:
        type: BareMetal
---
apiVersion: operator.openshift.io/v1
kind: IngressController
metadata:
  name: hcp2
  namespace: openshift-ingress-operator
spec:
  domain: hosted.hcp2.lapthorn.me
  replicas: 1
  namespaceSelector:
    matchLabels:
      kubernetes.io/metadata.name: clusters-hcp2
  endpointPublishingStrategy:
    type: LoadBalancerService
    loadBalancer:
      scope: Internal
      providerParameters:
        type: BareMetal
```

After creation, annotate the LoadBalancer services so MetalLB assigns them IPs from the correct pools:

```
# Find the LB service names
oc get svc -n openshift-ingress -l ingresscontroller.operator.openshift.io/owning-ingre

# Annotate for the correct MetalLB pool
oc annotate svc router-hcp1 -n openshift-ingress \
  metallb.universe.tf/address-pool=pool-100 \
  metallb.universe.tf/loadBalancerIPs=10.10.100.10

oc annotate svc router-hcp2 -n openshift-ingress \
  metallb.universe.tf/address-pool=pool-200 \
  metallb.universe.tf/loadBalancerIPs=10.10.200.10
```

7.2 Update the Default IngressController

Exclude HCP namespaces from the default IngressController so routes aren't double-admitted:

```
oc patch ingresscontroller default -n openshift-ingress-operator --type=merge -p '{
  "spec": {
    "namespaceSelector": {
      "matchExpressions": [{
        "key": "kubernetes.io/metadata.name",
        "operator": "NotIn",
        "values": ["clusters-hcp1", "clusters-hcp2"]
      }]
    }
  }
}'
```

Part 8: Deploy Hosted Clusters

8.1 Create Secrets

Each hosted cluster needs a pull secret and an etcd encryption key.

```
# Pull secrets
oc create secret generic hcp1-pull-secret -n clusters \
  --from-file=.dockerconfigjson=pull-secret.json \
  --type=kubernetes.io/dockerconfigjson

oc create secret generic hcp2-pull-secret -n clusters \
  --from-file=.dockerconfigjson=pull-secret.json \
  --type=kubernetes.io/dockerconfigjson
```

Etcd encryption keys — must be exactly 16, 24, or 32 characters (AES-CBC requirement):

```
# openssl rand -base64 24 produces exactly 32 chars — correct!
# openssl rand -base64 32 produces 44 chars — WRONG, will fail

KEY1=$(openssl rand -base64 24)
KEY2=$(openssl rand -base64 24)

oc create secret generic hcp1-etcd-encryption-key -n clusters \
  --from-literal=key="$KEY1"
oc label secret hcp1-etcd-encryption-key -n clusters \
  hypershift.openshift.io/safe-to-delete-with-cluster=true

oc create secret generic hcp2-etcd-encryption-key -n clusters \
  --from-literal=key="$KEY2"
oc label secret hcp2-etcd-encryption-key -n clusters \
  hypershift.openshift.io/safe-to-delete-with-cluster=true
```

8.2 HostedCluster and NodePool Manifests

Create hcp1 (repeat with `hcp2`, `localnet-200`, `pool-200`, and `hcp2.*` hostnames):

```
apiVersion: v1
kind: Namespace
metadata:
  name: clusters
---
apiVersion: hypershift.openshift.io/v1beta1
kind: HostedCluster
metadata:
  name: hcp1
  namespace: clusters
spec:
  autoscaling: {}
  capabilities: {}
  configuration: {}
  controllerAvailabilityPolicy: SingleReplica
  dns:
    baseDomain: lapthorn.me
  etcd:
    managed:
      storage:
        persistentVolume:
          size: 8Gi
          storageClassName: lvms-vg1
        type: PersistentVolume
    managementType: Managed
  fips: false
  networking:
    clusterNetwork:
    - cidr: 10.132.0.0/14
    networkType: OVNKubernetes
    serviceNetwork:
    - cidr: 172.31.0.0/16
  olmCatalogPlacement: management
  platform:
    kubevirt: {}
    type: KubeVirt
  pullSecret:
    name: hcp1-pull-secret
  release:
    image: quay.io/openshift-release-dev/ocp-release:4.22.14-x86_64
  secretEncryption:
    aescbc:
      activeKey:
        name: hcp1-etcd-encryption-key
    type: aescbc
  services:
  - service: APIServer
    servicePublishingStrategy:
      type: LoadBalancer
  - service: OAuthServer
```

```
    servicePublishingStrategy:
      type: Route
      route:
        hostname: oauth-openshift.hosted.hcp1.lapthorn.me
  - service: Konnectivity
    servicePublishingStrategy:
      type: Route
      route:
        hostname: konnectivity.hosted.hcp1.lapthorn.me
  - service: Ignition
    servicePublishingStrategy:
      type: Route
      route:
        hostname: ignition.hosted.hcp1.lapthorn.me
  sshKey: {}
---
apiVersion: hypershift.openshift.io/v1beta1
kind: NodePool
metadata:
  name: hcp1
  namespace: clusters
spec:
  arch: amd64
  clusterName: hcp1
  management:
    autoRepair: false
    upgradeType: Replace
    replace:
      rollingUpdate:
        maxSurge: 0
        maxUnavailable: 1
  nodeDrainTimeout: 0s
  nodeVolumeDetachTimeout: 0s
  platform:
    kubevirt:
      additionalNetworks:
      - name: localnet-100
      attachDefaultNetwork: false
      compute:
        cores: 2
        memory: 9Gi
        qosClass: Burstable
      networkInterfaceMultiqueue: Enable
      rootVolume:
        persistent:
          accessModes:
          - ReadWriteOnce
          size: 40Gi
          storageClass: lvms-vg1
        type: Persistent
    type: KubeVirt
```

```
  release:
    image: quay.io/openshift-release-dev/ocp-release:4.22.14-x86_64
  replicas: 0
```

8.3 Deploy and Scale

```
# Apply manifests (with replicas: 0 so control plane starts first)
oc apply -f hcp1.yaml
oc apply -f hcp2.yaml

# Wait for control planes to become available
oc get hostedclusters -n clusters -w

# Annotate the kube-apiserver LoadBalancer services for MetalLB
oc annotate svc kube-apiserver -n clusters-hcp1 \
  metallb.universe.tf/address-pool=pool-100 \
  metallb.universe.tf/loadBalancerIPs=10.10.100.11

oc annotate svc kube-apiserver -n clusters-hcp2 \
  metallb.universe.tf/address-pool=pool-200 \
  metallb.universe.tf/loadBalancerIPs=10.10.200.11

# Scale worker VMs to 1
oc scale nodepool hcp1 -n clusters --replicas=1
oc scale nodepool hcp2 -n clusters --replicas=1
```

SNO NodePool strategy: `maxSurge: 0, maxUnavailable: 1`. On SNO there isn't enough memory to run old + new VMs simultaneously during rolling updates. This setting deletes the old VM before creating the new one (causes brief downtime during updates, which is acceptable for a PoC).

SNO VM memory: 9Gi, not 8Gi. With 8Gi, the `klusterlet-addon-workmgr` pod (MCE addon) can't schedule due to insufficient memory, which causes the MCE console to show "Update failing" / "Unknown" status.

8.4 Wait for Completion

```
# Watch hosted clusters reach Completed
oc get hostedclusters -n clusters -w

# Get kubeconfigs
oc extract -n clusters secret/hcp1-admin-kubeconfig --to=- --keys=kubeconfig > /tmp/hcp
oc extract -n clusters secret/hcp2-admin-kubeconfig --to=- --keys=kubeconfig > /tmp/hcp

# Get kubeadmin passwords
oc extract -n clusters secret/hcp1-kubeadmin-password --to=-
oc extract -n clusters secret/hcp2-kubeadmin-password --to=-
```

Part 9: Verify L2 Isolation

9.1 Test Cross-Network Connectivity (Should FAIL)

From the hcp1 worker VM, try to reach the hcp2 worker VM:

```
ssh core@<SERVER_IP>
KUBECONFIG=/tmp/hcp1-kubeconfig oc debug node/<HCP1_NODE> -- \
  chroot /host ping -c3 -W2 10.10.200.175
```

Expected output: `100% packet loss`. The iptables FORWARD DROP rules block all traffic between 10.10.100.0/24 and 10.10.200.0/24.

9.2 Verify DROP Counters

```
ssh core@<SERVER_IP>
sudo iptables -L FORWARD -n -v | grep DROP
```

You should see non-zero packet counts on the DROP rules, confirming traffic was blocked.

9.3 Test Same-Network Connectivity (Should PASS)

From a worker VM, ping its own gateway:

```
KUBECONFIG=/tmp/hcp1-kubeconfig oc debug node/<HCP1_NODE> -- \
  chroot /host ping -c3 10.10.100.1
```

Expected: `0% packet loss`.

Part 10: Access Hosted Cluster Consoles

The hosted cluster consoles are on the isolated data networks (apps.hcpX.lapthorn.me → 10.10.X00.12). They are not reachable from the public internet. Use a SOCKS proxy.

10.1 Start a SOCKS Proxy

On your workstation:

```
ssh -D 1080 -N core@157.180.56.29
```

This creates a SOCKS5 proxy on `localhost:1080` that tunnels through the SNO node.

10.2 Configure Your Browser

Firefox (recommended — supports per-browser proxy without affecting system):

1. Settings → General → Network Settings → Settings...
2. Select **Manual proxy configuration**
3. SOCKS Host: `127.0.0.1`, Port: `1080`
4. Select **SOCKS v5**
5. Check **Proxy DNS when using SOCKS v5** (important — DNS must resolve through the SNO node's dnsmasq)
6. Click OK

Chrome (alternative — use a separate profile or extension like SwitchyOmega):

```
google-chrome --proxy-server="socks5://127.0.0.1:1080" --user-data-dir=/tmp/chrome-sock
```

10.3 Access the Consoles

With the SOCKS proxy active:

Cluster	Console URL	Username
hcp1	`https://console-openshift-console.apps.hcp1.lapthorn.me`	kubeadmin
hcp2	`https://console-openshift-console.apps.hcp2.lapthorn.me`	kubeadmin

Accept the self-signed certificate warning. Log in with `kubeadmin` and the password from Part 8.4.

> **If the console doesn't load:** Check that "Proxy DNS when using SOCKS v5" is enabled. Without it, your browser resolves `apps.hcpX.lapthorn.me` via public DNS (which has no record for it) instead of through the SNO node's dnsmasq.

Troubleshooting

Worker VMs stuck in Provisioning

Check if the ignition endpoint is accessible:

```
ssh core@<SERVER_IP>
curl -sk https://ignition.hosted.hcp1.lapthorn.me
```

If you get a certificate error, the IngressController shard isn't admitting the ignition route. Verify the IngressController uses `namespaceSelector` (not `routeSelector`).

Ingress operator shows Degraded (canary check failures)

SNO: The ingress operator (on the management cluster) checks canary routes through the Konnectivity proxy → worker VM → apps VIP → back to same VM. This hairpin fails without the MASQUERADE rules from Part 6.2.

Verify the hairpin rules are in place:

```
sudo iptables -t nat -L POSTROUTING -n -v | grep MASQUERADE
```

You should see per-VM MASQUERADE rules for the router NodePorts.

MCE shows "Update failing" or "Unknown" status

Check if the `klusterlet-addon-workmgr` pod is Pending inside the hosted cluster:

```
KUBECONFIG=/tmp/hcp1-kubeconfig oc get pods -n open-cluster-management-agent-addon
```

If Pending with `Insufficient memory`, increase the VM memory in the NodePool (9Gi minimum).

CUDN changes not taking effect

The CUDN `network` spec is immutable. To change it: 1. Scale NodePools to 0: `oc scale nodepool hcp1 -n clusters --replicas=0` 2. Delete the CUDN: `oc delete`

`clusteruserdefinednetwork localnet-100` 3. Recreate with the new spec 4. Scale back: `oc scale nodepool hcp1 -n clusters --replicas=1`

dnsmasq fails after reboot

If dnsmasq fails with "unknown interface brdXp0", the OVS bridge interfaces haven't been created yet by NMState. Remove any `interface=` or `bind-*` directives from `/etc/dnsmasq.d/hcp-data.conf` and restart.

DNAT rules stop working after VM recreation

VM IPs change on every recreation. Re-run the commands from Part 6.2 with the new IPs. Check current IPs with:

```
oc get vmi -A -o wide
```

SNO-Specific Limitations Summary

Issue	Cause	Fix	Multi-node?
Hairpin NAT for apps ingress	MetalLB VIP and VM endpoint on same node	MASQUERADE iptables rules	Not needed — VIP on different node
maxPods 500	All HCP control plane pods on single node	KubeletConfig	May not be needed with pods spread
NodePool maxSurge: 0	Can't run old + new VM simultaneously	Delete-before-create strategy	Use default maxSurge: 1
VM memory 9Gi (not 8Gi)	work-manager addon can't schedule	Extra 1Gi headroom	Not needed with more nodes
Cross-network API access works	SNO owns all MetalLB VIPs locally (INPUT, not FORWARD)	Accept for PoC, add INPUT rules for prod	VIPs on separate nodes, naturally isolated